



PHASE 2

Architecture and Technology Stack

Layers, request lifecycle, project structure and technology choices

SMARTPOCKET AI • SMART BUDGET AND RECOMMENDATION ASSISTANT

1. Architecture

1.1 Logical Architecture

SmartPocket AI follows a three-tier client-server design with an external AI dependency. The browser manages forms and result rendering. FastAPI provides validation, authentication, orchestration, and REST endpoints. SQLite stores users and plan history. Gemini generates a candidate recommendation, while backend normalization produces the trusted response.

Layer (top to bottom)	Responsibility
Browser UI	Forms, authentication state, result cards
FastAPI Application	Validation, JWT checks, routing, orchestration
Gemini Service	Prompt construction and structured AI response
Normalization Layer	Recalculation, scaling, schema enforcement
SQLite Data Layer	Users and per-user recommendation history

1.2 Request Lifecycle

- The browser submits planner inputs and an authorization token.
- FastAPI validates the token and request schema.
- The AI service creates a constrained prompt and requests JSON output.
- The response is parsed, normalized, recalculated, and validated.
- The browser receives the clean response and renders summary and item cards.
- The plan can be stored as a history entry linked to the authenticated user.



1.3 Project Structure

```
project/
|-- main.py          # application entry point and router registration
|-- database.py      # SQLite engine and session factory
|-- models.py        # User and HistoryEntry ORM models
|-- schemas.py       # request and response validation models
|-- auth.py          # password hashing and JWT helpers
|-- gemini_service.py # prompt, model call, parsing, normalization
|-- routers/
|   |-- auth_router.py    # register, login, session information
|   |-- recommend_router.py # home, party and jewelry recommendations
|   |-- history_router.py  # create, list and delete history
|-- requirements.txt
|-- .env
`-- Frontend/
    |-- index.html, login.html, register.html, dashboard.html
    |-- home-planner.html, party-planner.html, jewelry-planner.html
    |-- history.html
    |-- css/style.css
    |-- js/app.js, recommend.js, icons.js
```

2. Technology Stack

Layer	Technology	Responsibility
Frontend	HTML5, CSS3, JavaScript	Forms, navigation, API calls and result rendering
API	FastAPI on Uvicorn	REST routes, validation, authentication and orchestration
Validation	Pydantic	Typed request and response contracts
AI	Google Gemini	Text and optional image-based recommendation generation
Persistence	SQLite + SQLAlchemy	User accounts and saved history
Authentication	JWT + bcrypt	Token-based access and password hashing
Configuration	.env	Externalized keys, secrets and allowed frontend origins

2.1 Technology Choices

FastAPI provides typed request validation and interactive API documentation. Pydantic defines strict data contracts. SQLAlchemy separates persistence logic from route handling. JWT enables stateless protected API calls, and bcrypt prevents plaintext password storage. Gemini is used as the generative recommendation component, but its arithmetic and output structure are not treated as authoritative.